Adding Order History to Your Experience

Table of Contents

Adding Order History to Your Experience

Last Updated: 07/21/2021

Retrieve a complete order history for your consumers.

> **TIP**: Before using this guide you should have already completed Adding Checkout to Your Experience (https://nde-devportal-docs.niketech.com/doc/commerce/checkout/use-checkout.html).

Introduction

Adding consumer order history to your app can be as simple as this two-step process:

1. Your application makes an User Order Summary API (https://developer.niketech.com/docs/projects/User%20order%20summary?tab=api) request to retrieve all, or a filtered list of a member's orders.

2. Using an order ID from the User Order Summary (https://developer.niketech.com/docs/projects/User%20order%20summary?tab=api) response, your application makes a request to the User Order Details API to list order details for a member or guest.

What is an Order?

An order consists of all data related to:

- Consumers' purchased items/services
- Consumers' payment method(s) and billing address(es)
- Consumers' shipping method(s) and shipping address(es)
- Pricing
- Taxes
- Promotions
- Status

An order is created in the last step of Checkout (https://nde-devportal-docs.niketech.com/doc/commerce/checkout/use-checkout.html) when the consumer has provided all the necessary checkout information and submits it for fulfillment. After an order is created, it is stamped with a unique order number, the date and time the order was submitted, and a status of CREATED. The order is assigned different statuses as it progresses through the order lifecycle. See Understanding Order Status for more detail.

User vs. Core APIs

In the two-step process described earlier, we utilized the User Order Summary and User Order Detail APIs. These APIs provide a subset of order history data to apps that communicate over the public internet, e.g. the Nike app or Nike.com.

If your use case requires the superset of order history data, and your app communicates inside the Nike network,

then there may be an additional option for you: the core Order Summary (https://developer.niketech.com/docs/projects/Order%20Summary%20Service?tab=api) and Order Detail (https://developer.niketech.com/docs/projects/Order%20Detail%20Service?tab=api) APIs. The core APIs provide data to apps in Nike Retail stores, and the Consumer Services Portal (CSP), for example.

> **NOTE**: Because the core APIs are a superset of what the user APIs offer, much of the content in this document applies equally to both sets of APIs. Where there are important differences between the core and user APIs, they will be specifically mentioned.

Key Terms

Listed below are key terms for the Order History APIs.

Table 1: Key Terms for Order History APIs

Term	Definition
Employee	An employee that is logged in with a Swoosh account
Guest	An anonymous consumer, i.e. not logged-in with a Nike account
Member	A logged-in consumer with a Nike account
Order	A set of data for a checkout submitted by a consumer, e.g. products/services, payment methods, shipping methods, taxes, and promotions
Order Details	Use either the core or user Order Details API to get full details of an order for a member or guest
Order Line	A line item with an order, representing a product, payment, etc. Found in the `lineItems` in the Order Details response
Order Number	The unique `id` for the order
Order Status	The status of an order, as represented for individual lines and payments, or as a combined `status`. See Understanding Order Status for more
Order Summary	Use either the core or user Order Summary API to get a list of orders for a Nike member
DOMS	Nike's Digital Order Management System
BOPIS	The 'Buy Online, Pickup In Store' scenario. Consumer buys products online from a nearby Nike

Term	Definition
	store's inventory, then does pickup at said Nike store (usually within a few hours)
PUP	'Pickup Points', where consumer buys online, then Nike ships the order to a pickup location of the consumer's choice
RESERVE	Consumer reserves products online from a nearby Nike store's inventory, then optionally purchases reserved products from said store
ShipToStore	Consumer buys products online, then Nike ships the order to a Nike store of the consumer's choice for pickup

Step 1: List a Member's Orders

Use the User Order Summary API (https://developer.niketech.com/docs/projects/User%20order%20summary?tab=api) to get all, or a filtered list of orders for a Nike member. By making their past orders available to members as a self-service in your app, they can view their product and payment history without having to contact Consumer Services.

> **TIPS**
>
> - User Order Summary returns limited information about each order. For order pricing, tax, shipping, and detailed product information, or if you want to list the details of a guest's order, see List Order Details for a Member or Guest.
> - For the required request headers, see Required Request Headers.

Customizing Your Results

You control what is returned in your result set, and how it is sorted, through URL parameters.

Filtering

The table below lists the fields by which you can filter your User Order Summary results. If no filter is applied, all of a member's orders are returned. While some filters only allow one value, you can send multiple filters in the same request. For instance, even though only one `orderSubmitDateAfter` filter value is allowed, you can request a User Order Summary filtered by `orderSubmitDateAfter` and status. Note that filter parameter names and values are case-sensitive.

Table 2: Available Filters for User Order Summary

Order Field Name	Description	Sample Value
`orderType`	List orders of this type. You can	SALES_ORDER

Order Field Name	Description	Sample Value
	only filter by one orderType at a time.	
`storeId`	List orders placed in this store. You can only filter by one storeId at a time.	28382
`status`	List orders with this status.	Cancelled
`orderSubmitDateAfter`	List orders placed after this date. You can only filter by one date at a time.	Format is yyyy-MM-dd'T'HH:mm:ssZ
`orderLines.parentSalesOrderNumber`	Allows search of return orders based on sales order number	O7280930876
`orderLines.parentSalesOrderLineKey`	Allows search of return orders based on sales order line key	20170822053845753136865 45
`email`	Filter by customer email address. Only works in combination with `orderLines.parentSalesOrderNumber` or `orderLines.parentSalesOrderLineKey` and it must match email in parent order	sample@gmail.com
`phoneNumber`	Filter by customer phone number. Only number 0-9 are valid. Only works in combination with `orderLines.parentSalesOrderNumber` or `orderLines.parentSalesOrderLineKey` and it must match phone number in parent order	7134567890

Sorting

You can sort a member's orders in several ways using the `sort` query parameter. You can sort by one or more order fields, separated by a comma. If the field name you want to sort by is nested, refer to it with dot notation. For sort parameter syntax, see the Query Parameters (https://nde-devportal-docs.niketech.com/doc/getting-started/using-nike-apis.html#query-parameters) section of Using Nike APIs (https://nde-devportal-docs.niketech.com/doc/getting-started/using-nike-apis.html).

> **TIP**: It is recommended that your app pass the `sort` query parameter in the request to ensure that the results are sorted appropriately for your experience.

Other Query Parameters

The User Order Summary API also supports the `fields`, `count` and `anchor` query parameters to restrict the results to certain fields, restrict the number of results, and control pagination. For more information on syntax, see the Query Parameters (https://nde-devportal-docs.niketech.com/doc/getting-started/using-nike-apis.html#query-parameters) section of Using Nike APIs (https://nde-devportal-docs.niketech.com/doc/getting-started/using-nike-apis.html).

User Order Summary Scenarios

Table 3: Scenarios for User Order Summary

I want to list for a member	Sample Query
Order types of RESERVE_ORDER, submitted after 2021-01-24, purchased at store 12345	`https://api.nike.com/order_mgmt/user_order_summary/v2?filter=storeId(12345)&filter=orderSubmitDateAfter(2021-01-24)&filter=orderType(RESERVE_ORDER)`
Orders with a status of `Shipped` or `Delivered`	`https://api.nike.com/order_mgmt/user_order_summary/v2?filter=status(Shipped,Delivered)`
All orders sorted in ascending modificationDate	`https://api.nike.com/order_mgmt/user_order_summary/v2?sort=modificationDateAsc`
Just order ID, status and submitted date fields of all orders	`https://api.nike.com/order_mgmt/user_order_summary/v2?fields=id,status,orderSubmitDate`
Two most-recently-submitted orders	`https://api.nike.com/order_mgmt/user_order_summary/v2?count=2`

Executing the Request

Sample CURL for User Order Summary for a Member/Employee:

```
curl -X GET \
https://api.nike.com/order_mgmt/user_order_summary/v2 \
-H 'Authorization: Bearer <your token here>'
```

Parsing the Response

The User Order Summary JSON response contains several fields relating to status. See Understanding Order Status for more detail about how status is determined, and the suggested order statuses to display in your experience. See the User Order Summary API (https://developer.niketech.com/docs/projects/User%20order%20summary?tab=api) for a full list of fields returned in the response.

Step 2: List Order Details for a Member or Guest

Use the User Order Details API (https://developer.niketech.com/docs/projects/User%20order%20details?tab=api) to get order details for a member or guest. This API returns a complete picture of an order including product detail, tax information and line item details. If you are looking for higher level order information, or you want information on more than one order for either a member or employee, see List a Member's Orders.

> **TIP**: The User Order Details API does not return image URLs, but you can call the Merchandised Product API (https://nde-devportal-docs.niketech.com/doc/commerce/product/use-merch-product.html#product-image-set-by-style-color) using the style-color returned from the User Order Details API to get a list of images for a styleColor and country.

The User Order Details API requires that you pass certain headers in the request depending upon whether the consumer is a member, guest, or employee. For more information, see Required Request Headers.

Required Request Parameters

For members and employees, the `orderNumber` path parameter is required. For validation purposes for guest consumers, the `orderNumber` path parameter and email address filter parameter are required.

> **TIPS**:
>
> - To avoid a 404 response, the authentication header for a member request must match the member who created the order.
> - For guests, the email address must match the shipTo email address on the order.

Customizing Your Results

You control what is returned in your result set through URL parameters. The User Order Details API supports the `fields` query parameter to restrict the fields returned in the response. Since the User Order Details API only returns one consumer order, the `anchor` , `sort` , and `count` query parameters are not supported. For more information on the `fields` query parameter syntax, see the Query Parameters (https://nde-devportal-docs.niketech.com/doc/getting-started/using-nike-apis.html#query-parameters) section of Using Nike APIs (https://nde-devportal-docs.niketech.com/doc/getting-started/using-nike-apis.html).

Filtering

The table below lists the fields by which you can filter your User Order Details results.

Table 4: Available Filters for User Order Summary

Order Field Name	Description	Sample Value
`email`	Filter by customer email address	sample@gmail.com
`phoneNumber`	Filter by customer phone number. Only numbers 0-9 allowed	7134567890

Let's take a look at some User Order Details (https://developer.niketech.com/docs/projects/User%20order%20details?tab=api) scenarios.

Table 5: Scenarios for User Order Details

I want to	Sample Query
List the details for order ID C00011554850 for a member	`https://api.nike.com/order_mgmt/user_order_detail/v2/C00011554850`
List the ID, status and shipping method fields for orderNumber C00011554850 for a guest	`https://api.nike.com/order_mgmt/user_order_details/v2/C00011554850?filter=email(my.email@address.com)&fields=id,status,orderLines.shippingMethod`

Executing the Request

Sample CURL for User Order Details C00011554850 for a member/employee:

```
curl -X GET \
https://api.nike.com/order_mgmt/user_order_detail/v2/C00011554850 \
-H 'Authorization: Bearer <your token here>'
```

Sample CURL for User Order Details C00011554850 for a guest:

```
curl -X GET \
'https://api.nike.com/order_mgmt/user_order_details/v2/C00011554850?filter=email%28my.email@address.com%29' \
-H 'X-Nike-Visitorid: 2c83877b-10fa-44da-a92d-2451efef8671' \
-H 'appid: com.nike.sport.running.ios' \
-H 'x-nike-visitid: 2'
```

Parsing the Response

The User Order Details (https://developer.niketech.com/docs/projects/User%20order%20details?tab=api) JSON response contains several fields relating to status. See Understanding Order Status for more detail about how status is determined, and the suggested order statuses to display in your experience. See the User Order Details API (https://developer.niketech.com/docs/projects/User%20order%20details?tab=api) for a full list of fields returned in the response.

Understanding Order Status

An order contains three types of statuses:

- Order
- Order line
- Payment

An order line is a Nike service or product associated with a quantity, e.g. Nike Air VaporMax, quantity 1. Orders have at least one order line and may have several. Each order line has one or more statuses that map(s) to a numeric status code. Behind the scenes, the status of the order is calculated by evaluating which order line has a `rolledUpStatus` with the highest status code.

The table below describes each status associated with an order.

Table 6: Fields Relating to Order Status for User Order Details/Summary

Status Field Name	Description	API
`status`	Status of the order. Matches the orderLines.rolledUpStatus with the highest status code on the order.	User Order Summary User Order Details
orderLines. `rolledUpStatus`	Status of an order line. Computed by “Partially” + orderLines.maxOrderLineStatus. e.g. “Partially Shipped”.	User Order Summary User Order Details
orderLines. `maxOrderLineStatus`	Status of the highest status code on this order line e.g “Shipped”. Matches orderLines.rolledUpStatus. This status is included in orderLines.statuses.	User Order Details
orderLines. `minOrderLineStatus`	Status of the lowest status code on this order line e.g. “Factory Delayed”. This status is included in orderLines.statuses.	User Order Details
orderLines. `statuses`	An array of statuses for each status code on this order line. The description with the highest status code matches orderLines.maxOrderLineStatus. The description with the lowest	User Order Details

Status Field Name	Description	API
	status code matches orderLines.minOrderLineStatus.	
`paymentStatus`	Status of payment.	User Order Summary

In the scenario illustrated by the User Order Details response below, a consumer has purchased two order lines. One order line has shipped and has a `rolledUpStatus` of “Shipped”, and the other has been delayed at the factory and has a `rolledUpStatus` of “Factory Delayed”. Because the “Shipped” status has a higher status code than the “Factory Delayed” status code, the order status is “Partially Shipped”, calculated by “Partially” + the order line with the highest `rolledUpStatus` on the order.

```
 {
"status": "Partially Shipped",
...
"orderLines": [
        {
            ...

            "statuses": [
                {
                    "description": "Shipped",
                    "quantity": 1,
                    "date": "2021-01-30T12:16:51Z"
                }
            ],
            "rolledUpStatus": "Shipped",
            ...
            "maxOrderLineStatus": "Shipped",
            ...
            "minOrderLineStatus": "Shipped",
            ...
        },
        {
            ...

            "statuses": [
                {
                    "description": "Factory Delayed",
                    "quantity": 1,
                    "date": "2021-01-30T12:16:51Z"
```

```
                }
            ],
            "rolledUpStatus": "Factory Delayed",
            ...
            "maxOrderLineStatus": "Factory Delayed",
            ...
            "minOrderLineStatus": "Factory Delayed",
            ...
        }
    ]
}
```

Let's look at a slightly more complex example. As shown in the User Order Details response below, a consumer purchased three identical shorts. One short was delivered and has a status of "Delivered", another short was delivered and returned and has a status of "Return Processed", and one short was shipped and has a status of "Shipped". Because the "Delivered" status has the highest status code of the order line, the `rolledUpStatus` of the order line is "Delivered". The highest `rolledUpStatus` of the order is "Delivered", so the order status is "Partially Delivered".

```
  {
"status": "Partially Delivered",
...
"orderLines": [
        {
            ...

            "statuses": [
                {
                    "description": "Delivered",
                    "quantity": 1,
                    "date": "2021-01-30T12:16:51Z"
                },
                {
                    "description": "Shipped",
                    "quantity": 1,
                    "date": "2021-01-30T12:16:51Z"
                }
                {
                    "description": "Return Processed",
                    "quantity": 1,
                    "date": "2021-01-30T12:16:51Z"
                }
```

```
            ],
            "rolledUpStatus": "Partially Delivered",
            ...
            "maxOrderLineStatus": "Partially Delivered",
            ...
            "minOrderLineStatus": "Shipped",
            ...
        }
    ]
}
```

Listed below are the order line statuses, status codes, and simple status:

- **STATUS**: The order line status is returned by both the User Order Summary and User Order Details APIs.
- **STATUS CODE**: Behind the scenes, status code is used by both APIs to calculate status ranking. However, the status code is not returned by either the User Order Summary or User Order Details API.
- **SIMPLE STATUS**: The simple status is not returned by either the User Order Summary or User Order Details API but is provided as a sample, consumer-friendly status. Your experience could perform a similar status-to-simple-status mapping to display status to the consumer.

Table 7: Matrix of Order Line Statuses, Status Codes, and Simple Statuses

STATUS	STATUS CODE * not returned by either API	SIMPLE STATUS * not returned by either API
Draft Order Created	1000	CREATED
Draft Order Reserved Awaiting Acceptance	1040	CREATED
Draft Order Reserved	1050	CREATED
Created or CREATED	1100	CREATED
Authorized	1100.01	CREATED
Not Authorized	1100.02	CREATED
Pending Completion	1100.03	CREATED
Origin Scan	1100.050	CREATED
In Transit	1100.100	CREATED
Acknowledged	1100.110	IN PROCESS
Work Order Cancelled	1100.1600	IN PROCESS
Factory Processing	1100.600	IN PROCESS

STATUS	STATUS CODE * not returned by either API	SIMPLE STATUS * not returned by either API
Factory Completed	1100.700	IN PROCESS
Factory Delayed	1100.800	IN PROCESS
Reserved Awaiting Acceptance	1140	IN PROCESS
Reserved	1200	IN PROCESS
Being Negotiated	1230	IN PROCESS
Accepted	1260	IN PROCESS
Rescheduling	1300	IN PROCESS
BOReview	1300	IN PROCESS
Unscheduled	1310	IN PROCESS
Scheduled	1500	IN PROCESS
Awaiting Chained Order Creation	1600	IN PROCESS
Awaiting Procurement Purchase Order Creation	2030	IN PROCESS
Awaiting Procurement Transfer Order Creation	2060	IN PROCESS
Chained Order Created	2100	IN PROCESS
PO Released	2100.100	IN PROCESS
Released For SO Create	2100.1000	IN PROCESS
DSV Acknowledged	2100.1100	IN PROCESS
Released To Third Party	2100.1200	IN PROCESS
Received By Partner	2100.1210	IN PROCESS
Confirmed By Partner	2100.1220	IN PROCESS
Order Processing	2100.1230	IN PROCESS
Order Delayed	2100.1250	IN PROCESS
PO Acknowledged	2100.200	IN PROCESS
Released To FC	2100.30	IN PROCESS

STATUS	STATUS CODE * not returned by either API	SIMPLE STATUS * not returned by either API
PO Sent To Warehouse	2100.300	IN PROCESS
PO Warehouse Acknowledged	2100.350	IN PROCESS
PO Scheduled For Pick	2100.40	IN PROCESS
Order Received	2100.400	IN PROCESS
PO Awaiting Shipment	2100.50	IN PROCESS
Order Processed	2100.500	IN PROCESS
DSV Acknowledged	2100.550	IN PROCESS
Factory Processing	2100.600	IN PROCESS
Sent For SO Create	2100.620	IN PROCESS
DSV Acknowledged	2100.650	IN PROCESS
Factory Completed	2100.700	IN PROCESS
Factory Delayed	2100.800	IN PROCESS
Procurement Purchase Order Created	2130	IN PROCESS
Procurement Purchase Order Shipped	2130.01	IN PROCESS
Work Order Created	2140	IN PROCESS
Work Order Completed	2141	IN PROCESS
Procurement Transfer Order Created	2160	IN PROCESS
Procurement Transfer Order Shipped	2160.01	IN PROCESS
Released	3200	IN PROCESS
Awaiting Shipment Consolidation	3200.01	IN PROCESS
Awaiting WMS Interface	3200.02	IN PROCESS
Published To WMS	3200.03	IN PROCESS
Acknowledged	3200.100	IN PROCESS

STATUS	STATUS CODE * not returned by either API	SIMPLE STATUS * not returned by either API
Sent To Warehouse	3200.1000	IN PROCESS
Released To Third Party	3200.1200	IN PROCESS
Received By Partner	3200.1210	IN PROCESS
Confirmed By Partner	3200.1220	IN PROCESS
Order Processing	3200.1230	IN PROCESS
Pending Release To SAP	3200.150	IN PROCESS
Scheduled For Pick	3200.200	IN PROCESS
Awaiting Shipment	3200.300	IN PROCESS
Order Received	3200.400	IN PROCESS
Order Processed	3200.500	IN PROCESS
Factory Processing	3200.600	IN PROCESS
Factory Completed	3200.700	IN PROCESS
DSV Acknowledged	3200.900	IN PROCESS
Sent To Node	3300	IN PROCESS
Included In Shipment	3350	IN PROCESS
Awaiting Invoice	3700.0101	IN PROCESS
Invoiced	3700.999	IN PROCESS
Held	8000	IN PROCESS
Carried	1100.7777	COMPLETE
Order Completed	2100.1240	COMPLETED
Order Completed	3200.1240	COMPLETED
Completed	3700.12	COMPLETED
Order Completed	3700.777710	COMPLETED
Order Completed	3700.7777.10	COMPLETED
Backordered From Node	1400	BACKORDER
Order Delayed	3200.1250	DELAYED

STATUS	STATUS CODE * not returned by either API	SIMPLE STATUS * not returned by either API
Factory Delayed	3200.800	DELAYED
Shipped	3700	SHIPPED
Origin Scan	3700.10	SHIPPED
Customs Cleared	3700.100	SHIPPED
Manifest Scan	3700.200	SHIPPED
In Transit	3700.30	SHIPPED
Delivered To Store	3700.105	DELIVERED
Notionally Delivered	3700.69	DELIVERED
Delivered	3700.70	DELIVERED
Order Delivered	3700.7777	DELIVERED
Ready For Pickup	3700.110	READY FOR PICKUP
Consumer Picked Up	3700.120	PICKED UP
Picked Up	3700.20	PICKED UP
Shipment Delayed	3700.130	SHIPMENT DELAYED
Shipment Delayed	8500	SHIPMENT DELAYED
Abandoned	3700.210	ABANDONED
Delivery Attempt 1	3700.40	DELIVERY ATTEMPT 1
Delivery Attempt 2	3700.50	DELIVERY ATTEMPT 2
Delivery Attempt 3	3700.60	DELIVERY ATTEMPT 3
Unable To Deliver	3700.80	UNABLE TO DELIVER
Refused Shipment	3700.90	REFUSED SHIPMENT
Pending Cancel	1300.9000	CANCEL IN PROCESS
Pending Cancel	3200.9000	CANCEL IN PROCESS
Cancelled	9000	CANCELLED
Void	9000.100	CANCELLED
Post Voided	9000.200	POST VOIDED

STATUS	STATUS CODE * not returned by either API	SIMPLE STATUS * not returned by either API
Delivered to Return Center	1100.200	RETURN CREATED
Early Refund	1100.300	RETURN CREATED
Return Created	3700.01	RETURN CREATED
Return Request Created	3700.011	RETURN CREATED
Return Received	3700.02	RETURN CREATED
Returned At Store	1100.400	RETURN COMPLETED
Returned At Store	3700.04	RETURN COMPLETED
Returned to DC	3700.85	RETURN COMPLETED
Returned To Carrier	3700.95	RETURN COMPLETED
Delivered to Return Center	1100.200100	RETURN IN PROCESS
Inspection Escalate	1100.500	RETURN IN PROCESS
Return In Progress	3700.03	RETURN IN PROCESS
Return Processed	3700.05	RETURN IN PROCESS
Return Invoiced	3950.01	RETURN IN PROCESS
Received at Return Center	3950.100	RETURN IN PROCESS
Inspection Passed	3950.200	RETURN IN PROCESS
Inspection Escalate	3950.300	RETURN IN PROCESS
Inspection Denied	3950.400	RETURN IN PROCESS
Inspected And Invoiced	3950.01.100	RETURN IN PROCESS
Return Denied	3700.06	RETURN DENIED
CSR Refund	3950.700	CSR REFUND
Inspected And Invoiced	3950.01100	RETURN PROCESSED
Exchange Order Created	3950.02.01	EXCHANGE CREATED
Awaiting Exchange Order Creation	3950.02	EXCHANGE IN PROCESS
CREATED		Created (Only for Reserve Orders)
UNRESERVED		UnReserved(Only for Reserve

STATUS	STATUS CODE * not returned by either API	SIMPLE STATUS * not returned by either API
		Orders)

API Quick Reference

Table 8: Order History Endpoints

Endpoint Name	Path	HTTP Method
User Order Summary (https://developer.niketech.com/docs/projects/User%20order%20summary?tab=api)	/order_mgmt/user_order_summary/v2	GET
User Order Details (https://developer.niketech.com/docs/projects/User%20order%20details?tab=api)	/order_mgmt/user_order_detail/v2/{orderNumber}	GET
Order Summary (https://developer.niketech.com/docs/projects/Order%20Summary%20Service?tab=api)	/order_mgmt/order_summary/v2	GET
Order Detail (https://developer.niketech.com/docs/projects/Order%20Detail%20Service?tab=api)	/order_mgmt/order_detail/v2	GET

Best Practices

Listed below are some best practices for working with User Order Summary and User Order Details.

Conditions for Retries

For all Nike Cloud APIs, the general rule is that HTTP 4XX error codes (except for 429) should not be retried, but HTTP 5XX errors can be retried. For general information on Nike error retry practices, see API Error Patterns (https://confluence.nike.com/display/NEA/API+Standards#APIStandards-Errors) on Confluence.

Testing

It is recommended to test all Order endpoints in the production environment as opposed to the test environment. Using the test environment can have unpredictable results, due to the many downstream dependencies required to provide 'production-like' responses.

Q: What are the boundaries for testing in production?

Performance tests at high volumes should **never** be done in production. All performance tests should be done in test.

Q: Is this service available in the test environment?

All services are available in the test environment, however, the data and services are not always available for end-to-end testing.

Q: Why are our integration tests (that use the test environment) failing?

When integrating for the first time, we can help ensure basic connectivity in the test environment before you deploy to production. However, we do not recommend relying on connections to the test environment on an ongoing basis, and provide no guarantees on the availability or retention of the data.

Teams should not introduce breaking changes in their contracts, so **mocking downstream dependencies** is often recommended to decouple development & testing between teams.

> **TIP**: Tools like WireMock (http://wiremock.org) allow you to mock out services for integration testing. Also, techniques like dark deployments & traffic shadowing can be used in Prod to validate new functionality.

Q: Why is an order not showing up in the test environment?

There are not as many system resources dedicated to the test environment, causing delays in asynchronous processing.

Caching Data

None of the endpoints described in this document support caching.

Troubleshooting

Here are some troubleshooting tips:

- Use the general troubleshooting tips in the Using Nike APIs (https://nde-devportal-docs.niketech.com/doc/getting-started/using-nike-apis.html#troubleshooting) guide.
- Use a Splunk query (requires access) to check for issues with your request.
- Contact the Orders team on the #mp-athena (https://nikedigital.slack.com/messages/C1H7ZM7J4) Slack channel for assistance.

Terms of Service

Following are the terms of service for the Order History APIs.

Authorization

Access Tokens

Calls to the User Order Summary and User Order Detail APIs require an access token be sent in the request header. This allows Nike to verify that your app is authorized to perform the action on behalf of the user.

Access tokens are obtained by calling Nike Unite services prior to calling the API which you ultimately want to reach.

To find out more on how to call Unite services to obtain access tokens, see the Authorization section of the Using Nike APIs (https://nde-devportal-docs.niketech.com/doc/getting-started/using-nike-apis.html#authorization) guide.

User Types

The Order APIs support 3 distinct user types:

- Member: user has logged in with their Nike account credentials
- Guest: user has not logged in (anonymous user)
- Employee: user is an employee of Nike or a subsidiary and has logged in with swoosh.com credentials (also known as Swoosh user type)

Required Request Headers

Listed below are the required request headers, which vary based on user type. Since most User Order Summary and User Order Details requests come through the Nike Edge router, these header values will be set automatically, provided your app calls the Unite services first to get an access token and passes that token in the request.

Table 9: Required Order History Request Headers by User Type

Header Name	Description	Member	Guest (User Order Details only)	Employee
Accept	Content type you will accept in response, application/json is only value allowed	X	X	X
Content-Type	Content type of the request, application/json is only value allowed	X	X	X
Authorization	Your access token	X	X	X

Header Name	Description	Member	Guest (User Order Details only)	Employee
	in the format of `Bearer {token}` indicating the consumer is logged in			
x-nike-visitorid	Unique identifier for the guest, validated by the Edge router and passed through to the service. Applies only to User Order Details API.		X	
x-nike-visitid	Integer identifying the guest's session. Applies only to User Order Details API.		X	
appId	Application making the API request e.g. com.nike.sport.running.ios	X	X	X

> **TIP:** For the Authorization header, use the token for the consumer's login session that you obtained from Nike Unite/Identity, prefixed by `Bearer` (note the single space after Bearer). This is necessary for Nike to verify that your app is authorized to perform the requested operation on behalf of the consumer.

See the User Types section of the Using Nike APIs (https://nde-devportal-docs.niketech.com/doc/getting-started/using-nike-apis.html#user-types) guide for more information.

JSON Web Token

The core Order Summary and Order Details endpoints require the additional authorization of a JSON Web Token (JWT). For more, see the JWT section of Using Nike APIs (https://nde-devportal-docs.niketech.com/doc/getting-started/using-nike-apis.html#jwt-json-web-token).

The User Order Summary and User Order Details endpoints do not require JWT authorization.

Sample Requests

Sample requests included throughout this guide contain unique IDs and access tokens that are spent/expired in the Production environment, so you will not be able to use them as-is for testing purposes. Reuse what you can and replace with valid IDs/access tokens when necessary.

Common Questions

Below are some commonly-asked questions for the Order APIs, grouped by topic.

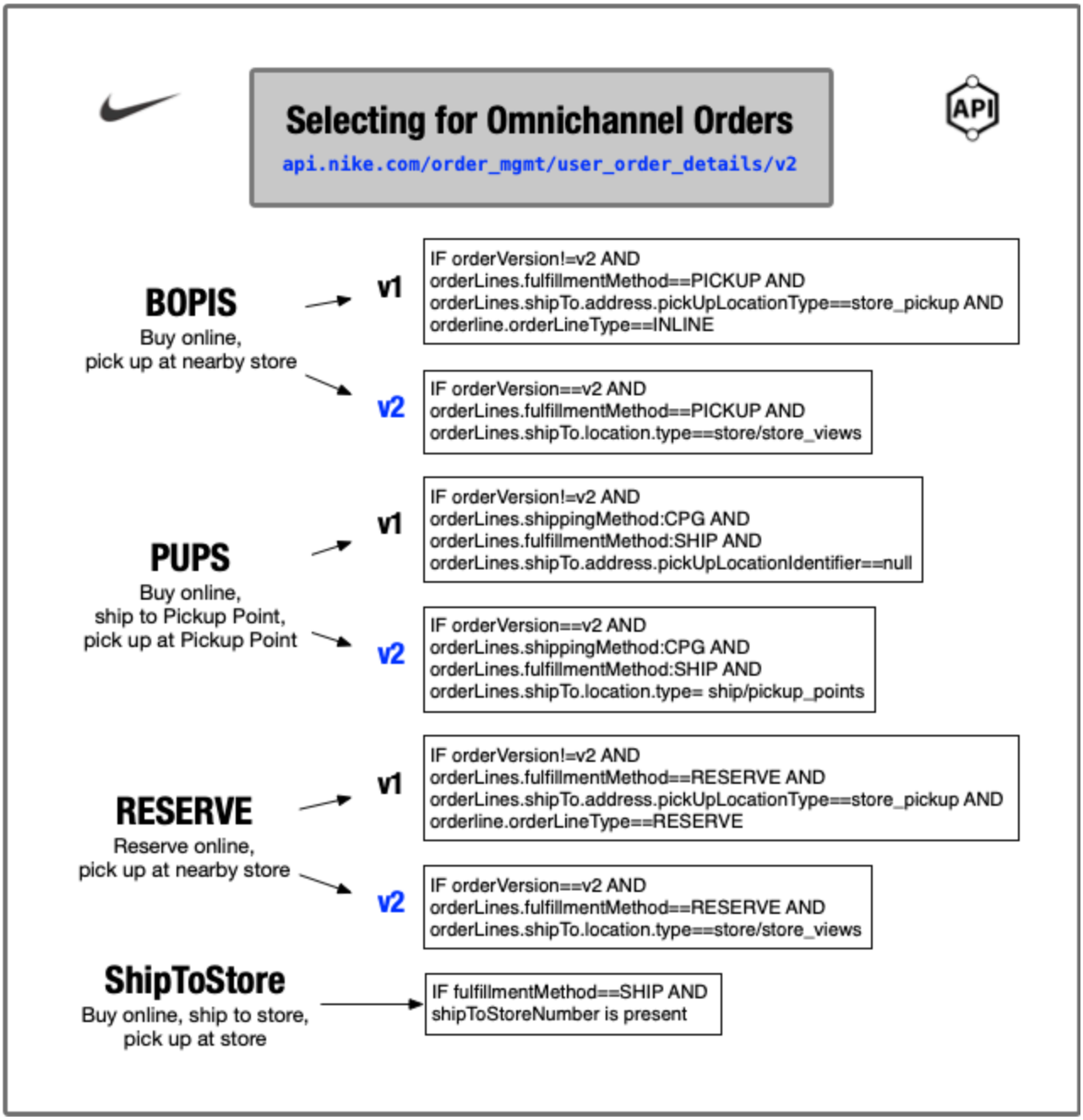
Identifying Types of Orders

Q: How do we identify order lines that have Nike By You (i.e. customized) products?

To identify Nike By You products, check that `orderLine` contains a `customizedProductReference` with a value in it.

Q: How do we identify omnichannel orders (BOPIS, PUPs, RESERVE, ShipToStore)?

The following graphic describes how to select for various types of omnichannel orders:



> **TIP:** For v1 payloads, the store address will be present in `orderLines.shipTo.address`. For v2 payloads, call the Store Views API (https://developer.niketech.com/docs/projects/Store%20Views%20V2?tab=api) with the value from `orderLines.shipTo.location.id` to get the address.

Q: How do we identify different types of cancelled or voided orders?

Each `orderLine` can contain a `cancellationReason`, `lineNotes`, and a `storeOrderLineDetail` object. The `cancellationReason` expresses why the orderLine was cancelled. A `lineNotes` object will contain a `reasonCode` field. The `storeOrderLineDetail` will contain a `transactionType` with the value of VOID.

Q: How do we distinguish between pure B&M, mixed cart, and pure digital orders?

Two key identifiers to understand the classification of an order are `orderClassification` and `orderLineType`.

`orderClassification` values describe the kind of order. Here are some examples:

Table 10: Example orderClassification Values

Value	Description
STANDARD	Digitally fulfilled orders from APPS, Web, Partners
STORE	Order created in a retails store - Point of Sale as well as Mobile point of sale
CONV_STORE	An order paid by Konbini or Convenience store mode of payment
CSRORDER	Order created by consumer services from Internal L3 menu
COD	Cash on Delivery order

Please see List of Values included in the Order Classification Field (https://confluence.nike.com/display/MOM/List+of+Values+included+in+the+Order+Classification+Field) for other order classifications:

The values in `orderLines.orderLineType` describe the type of order line, e.g. “GC” for gift card. See below for an incomplete list of `orderLineType`.

Examples:

- INLINE - Order placed online.
- STORE - Order placed in store.

Please leverage the combination of `orderClassification` and `orderLineType`.

Q: What fields do I use to identify store transactions?

Store information is at the order level for both v1 and v2:

- store.registerNumber
- store.storeNumber
- store.storeId
- store.transactionBeginDate
- store.transactionEndDate
- store.transactionNumber
- store.customerZipCode
- store.firstSalesPostingRequired

> **TIP**: `storeId` can also be used to call the Store Views API (https://developer.niketech.com/docs/projects/Store%20Views%20V2?tab=api).

Q: How do we identify where the order was captured from (i.e. the origin of the order?)

- If the order payload matches the v1 schema:
  - Use `originApplication` field (e.g. com.nike.commerce.checkout.web, com.nike.commerce.snkrs.web, com.nike.sport.running.ios)
- If the order payload matches the v2 schema:
  - Use `channel` field (e.g. nike.digital.web, nike.omega.web)

Q: How do we identify employee purchases?

To identify orders placed by employees, the order must be created using their respective Swoosh profile.

- If the order payload matches the v1 schema:
  - The division field will have a value of 55
- If the order payload matches the v2 schema:
  - The division field will have a value of 55
  - userInfo.userType field will have the value of nike:swoosh.

Q: How do we identify a cancelled quantity on an order?

Cancelled orders will have “Cancelled” in the `orderLines.statuses.description` field, and a non-zero integer in the `orderLines.statuses.quantity` field, for example:

```
"statuses": [
{
  "description": "Cancelled",
  "quantity": 1,
  "date": "2021-11-11T05:38:46.000Z"
}
```

Q: How do I distinguish between exchange and return orders?

For returns orders, the `orderType` would be equal to RETURN_ORDER. For exchange orders the orderType will be SALES_ORDER, and the `parentReturnOrder` will be populated.

Q: Is there a way to search for orders using the consumer’s gift card number?

No, this is not supported.

Q: How do I obtain Estimated Delivery Date (EDD) info?

- If the order payload matches the v1 schema:
  - The EDD information can be obtained from `orderLines.estimateDeliveryDate` field.
- If the order payload matches the v2 schema:
  - The EDD information can be obtained from `orderLines.estimatedDelivery` object.

Q: What fields do I use to identify an order with a pickup location?

- If the order payload matches the v1 schema:

- `orderLines.shipTo.address.pickUpLocationIdentifier`
- `orderLines.shipTo.address.pickUpLocationType`
- `orderLines.shipTo.address.pickUpLocation`

- If the order payload matches the v2 schema:
  - `orderLines.shipTo.address.pickUpLocationIdentifier`
  - `orderLines.shipTo.address.pickUpLocationType`

Q: How do I retrieve Converse orders?

To retrieve Converse orders, call User Order Details with value “converseus” in the optional `appId` request header. This also works for User Order Summary.

General Setup & Configuration

Q: Can I call the two User Order APIs if my app is hosted in an Amazon Web Services VPC?

Yes. The User Order APIs are exposed publicly, so it does not matter where you are calling from. If you are calling repeatedly from a small set of IP addresses, it might be possible that Nike’s bot-mitigation tools could interfere with your ability to make calls. If you are having issues, reach out to Slack channel #mp-athena (https://nikedigital.slack.com/messages/C1H7ZM7J4) for help.

Q: How do I get my appId on the allowed list?

Reach out to @athena-integration-support in our #mp-athena-integration-support Slack channel to get help with adding your appId to the allowed list.

Q: What is the vip address for User Order Details?

The vip address for User Order Details is order_mgmt-order_detail-v2.

Q: Does User Order Details purge any data?

User Order Details does not purge data. This service has data from February 2012 onward.

Q: How do we correlate Nike sales order numbers to partner order numbers?

`marketplaceDetail` is an object at the order level which contains information regarding the integrator/partner order number.

General Schema Questions

Q: What are the major differences between the User Order Details v1 and v2 schemas?

User Order Details v2 responds with orders that match either the v1 or v2 (Source Aware) schema. To see the differences between the two payloads, please see Analysis - Source Aware - Order Repo changes (https://confluence.nike.com/pages/viewpage.action?spaceKey=CE&title=Analysis+-+Source+Aware+-+Order+Repo+changes)

Q: Where do I find the total price of an order?

Both User Order Details and User Order Summary include the total price of an order in the `totalAmount` field:

```
"totalAmount": 100,
```

User Order Details also sends the total in `orderTotalDetails.orderTotal` :

```
"orderTotalDetails": [
  {
    "orderTotal": 100
  }
],
```

Q: Is the time in UTC or local time?

All times in Order History will be in UTC Zulu with no offset, e.g. 2021-03-20T01:06:53Z.

Q: What events trigger the status of an order to be updated?

Order Management has series of life cycle events, and there are also modifications triggered by customers, CSP, and Track and Trace. See Order Status Mapping for Consumers (https://confluence.nike.com/display/MOM/Order+Status+Mapping+for+Consumers) for more information regarding status.

Q: What are XPO orders?

XPO orders allow Nike to ship product(s) between its stores.

Questions About Returns

Q: Under what conditions will `parentSalesOrderNumber` be populated?

`parentSalesOrderNumber` will be populated for the following scenarios:

- Return orders
- Global store orders and store orders
  - Store orders can also have return lines

Q: Can I used `parentReturnOrder` to link the original sales order to return order?

No, `parentReturnOrder` is only used for exchange orders. If you wish to link the original sales order number to the return order number, you will have to leverage an User Order Summary collection call.

Q: How do we link sales orders to return orders?

Each return order has a `parentSalesOrderNumber` that is a one-way link from the return order to sales order. If you want to acquire all return orders associated with a sales order, call User Order Summary with a `parentSalesOrderNumber` as a filter.

Questions About Discounts, Taxes, and Promotions

Q: How do we identify discounts at both the header and line level?

The discounts at header level or line level can be identified using the corresponding `chargeCategory` at header/line level. Charge Category Discount details page provides all the possible ENUM values for the different

charge categories, and whether each charge category is discounted or not.

Q: Where are promotions located, and are they removable after having been applied to the order line?

Promotions, if any, will be present in `orderLines.promotions`, and cannot be removed from an order line.

Q: What if there is a cancellation on an order line? Would the `orderLines.promotions` section be updated?

No, the promotion will still be applied, regardless of order status.

Q: What do the VAT and NON-VAT field values mean?

VAT (Inclusive tax)

- object.headerTax and object.orderLines.lineTaxes.
  - effectiveTaxPercentage
  - effectiveTaxAmount

Non-VAT (Exclusive tax)

- object.headerTax and object.orderLines.lineTaxes.
  - taxPercentage
  - taxAmount

Q: How do I get invoice information?

Order Details has invoice identifiers in `chargeTransactionDetails.invoiceCollectionDetails.invoiceNumber`. With this value, a call can be made to the Order Invoice API (https://developer.niketech.com/docs/projects/Order%20Invoice%20API?tab=api).

Q: How do I identify EMEA fiscal fields?

Check the User Order Details v2 response for `additionalAddress.addressType` == FISCAL, and check if the nested `fiscalInformation` object is present.

`fiscalInformation` can contain the following EMEA fiscal fields:

- customer
  - customerType
  - vatNumber
  - profession
  - customerNumber
  - lotteryNumber
- invoice
  - invoiceType
  - invoiceNumber
  - sequenceNumber

Contacting the Team

Need to contact the Orders team?

Slack	#mp-athena
Confluence Space	Order Management (https://confluence.nike.com/display/CE/Order+Management#OrderManagement-CSP)
Team Contacts	**Intake, new requirements, onboarding, troubleshooting** Team Athena Lst-CE.Athena@nike.com
Intake	Please fill out an intake form (https://confluence.nike.com/display/MDPM/Intake+Form+for+Inventory+Management%2C+Order+Management+and+Digital+Fulfillment).

Document Change Log

Summary	Date
Initial publish	10/25/2021
Updated for v2 of both endpoints, updated doc format	02/17/2021
Added Common Questions section & content	07/21/2021

Next Steps

You've learned how to add Order History to your experience. Here are some next steps.

- Using Nike APIs (https://nde-devportal-docs.niketech.com/doc/getting-started/using-nike-apis.html)
- Glossary (https://nde-devportal-docs.niketech.com/doc/commerce/reference/glossary.html)